

Pinellas House Price Prediction Model

Spenser Johnson

Introduction

This report is on the creation of a prediction model for house prices in Pinellas County, Florida. Differences in house prices can seem at times objective and other times arbitrary, as seen by the two following examples.

 \$162,900 3 bds 2 ba 1,368 sqft - Home for sale 2001 83rd Ave N LOT 4030, Saint Petersburg, FL... VILLAGE GREEN	 \$69,000 3 bds 2 ba 1,392 sqft - Home for sale 34611 Pine Dr N, Pinellas Park, FL 33781 MH RESALES
--	--

The two houses in the example above are both 3 bedroom, 2 bathroom, 1300 square foot homes that are less than 2 miles apart as the crow flies. The pictures suggest that both are manufactured homes. So why is one house asking for more than twice the value of the other.

 \$350,000 2 bds 2 ba 1,125 sqft - House for sale 4901 Venetian Blvd NE, Saint Petersburg, FL... KELLER WILLIAMS ST PETE REALTY	 \$345,950 4 bds 3 ba 2,011 sqft - House for sale 1810 Maryland Ave NE, Saint Petersburg, FL 33703 STOFEL & ASSOCIATES REALTY
---	--

The two houses here are physically different. One is a single story, with 2 bedrooms, 2 bathrooms, 1125 square feet and a pool, with an asking price of \$350,000. The other house is

two story, 4 bedrooms, 3 baths, 2011 square feet with an asking price for less than the smaller house. These two houses are within 1000 feet of each other.

For most purchases, you would expect similar items to have similar prices. You would expect a larger amount of an item purchased to cost more. Yet in the examples above, the two most similar houses are far apart in price and the smallest house has the largest price. This report goes into detail on determining which factors and features have the greatest effect on the median house price, and goes through the steps of building a machine learning model to predict house prices based on those features.

Data Wrangling

Possible Factors

Housing prices have lots of factors. Local housing supply. Size of the house and lot. Style of construction. Location, location, location. Most people looking to buy a house start with the budget they think they can handle, then look for a house that meets their needs, hoping the house of their dreams is in their price range. The number of bedrooms and bathrooms can be a concern for a new couple looking to start a family or a couple of empty-nesters looking to downsize. Some people might prefer new construction while others will want a house that might need some d-i-y fixing up.

Data Sourcing

A starting source for data was Dolthub, a version-controlled database. It had a couple of dataset that contained housing sales from across the country. Reviewing what the dataset had for Pinellas County, many fields were blank and the data was not as feature-rich as desired. Following the source of the data, I found the Pinellas County Property Assessor's Office.

The Pinellas County Property Assessor's Office(PCPAO) website has an advanced property search to see records of all properties in Pinellas County. PCPAO data was much more feature rich, consisting of location information, construction information, lot information, tax and assessment data, and dates of each sale. Data was collected from the Pinellas County Property Assessor's Office Website. It consisted of 1.15 million rows of property sales to begin with, focusing on all sales of properties with residential use code 0110-Single Family Home.

Cleaning

I started data cleaning by making sure that the 'PROPERTY_USE' was specifically '0110 Single Family Home' and narrowing the pandas dataframe to include only that type of property use. I dropped the 'PARCEL_TYPE' column as all the properties were residential in nature. I then proceeded to drop columns that were not predictive of housing price. These were owner name columns and address information columns other than city and zip code. Other columns that were dropped include: NBORHOOD_CD, TAX_DIST_DSCR, and SPECIAL_ASSESSMENT. Those three columns were all tax-related, either identifying comparable houses for tax purposes, describing the tax district of the house, and identifying special rules for taxing a house. Since tax rules on each house get an update after sale, those columns would not help in predicting sale prices.

Some rows were dropped during the inspection of some columns. While reviewing TOTAL_LIVING_UNITS and LIVING_UNITS, I found that some records had values other than 1. Since I was working with single-family homes, I decided to drop the rows that had a value other than 1 for either of those columns. This led to dropping about 800 rows. After the TOTAL_LIVING_UNITS and LIVING_UNITS columns were both 1, I then dropped the columns.

I made a similar choice for DLHL_YN, a column that identifies whether a house is a designated local historical landmark. I dropped the 276 rows that had 'Yes', then dropped the column.

The FRONTAGE and VIEWS columns had many null values and were dropped. LAND_SIZE was dropped as it was a written description of each lot's width and depth. While these values would have been interesting, I already had LAND_AREA_SF. I didn't want to try and break down the width and depth into their own columns and it would have been unclear which measurement faced the road. ACREAGE was also dropped as it correlated perfectly with LAND_AREA_SF, and the latter was in the same units as other columns. The QU_FLG column was also dropped as all the sales by this point were qualified sales.

With the variety of construction features, I had to do some cleaning. I turned the 'COOLING' feature into a 'CENTRAL_COOLING' yes/no column. For other construction features, I cleaned text values by replacing spaces in the text with single underscores.

'EFFECTIVE_AGE' was a numerical description of how old the house was, considering any remodels and repairs done. 'YEAR_BUILT_BLD1' gave the year the house was built. I turned the column into a 'REAL_AGE' column by subtracting the value from 2024.

Checking my progress, I had narrowed the data from 1.15 million rows and 53 columns down to 700 thousand records and 33 columns. Now I limited the data to each parcel's most recent sale. Checking one parcel, my dataset found it was sold 6 times in the same year. Checking on the PCPAO website, the same parcel was only sold 3 times in that year. I found that my dataset had duplicated rows. I dropped duplicated rows, keeping the last one of each duplicate. Then I sorted the properties by sales date and dropped rows subset by PARCEL_ID, keeping the last record of each parcel sale.

I then inspected the city and zip code columns as a pair. I found that some city/zip code combinations were rare, likely indicating an error with the city or zip. Rather than working through manually fixing the records, I dropped rows which had a city/zip pair with 30 or less occurrences.

Subdivisions was the final categorical column to inspect, having 2000 null values out of 218 thousand records. There were 6094 unique subdivision names. At least 20 subdivisions had only one house, while another 20 subdivisions had 350 houses or more. With the amount of categorical values to consider, which would later be turned into their own columns, I made the choice to drop the subdivisions column entirely.

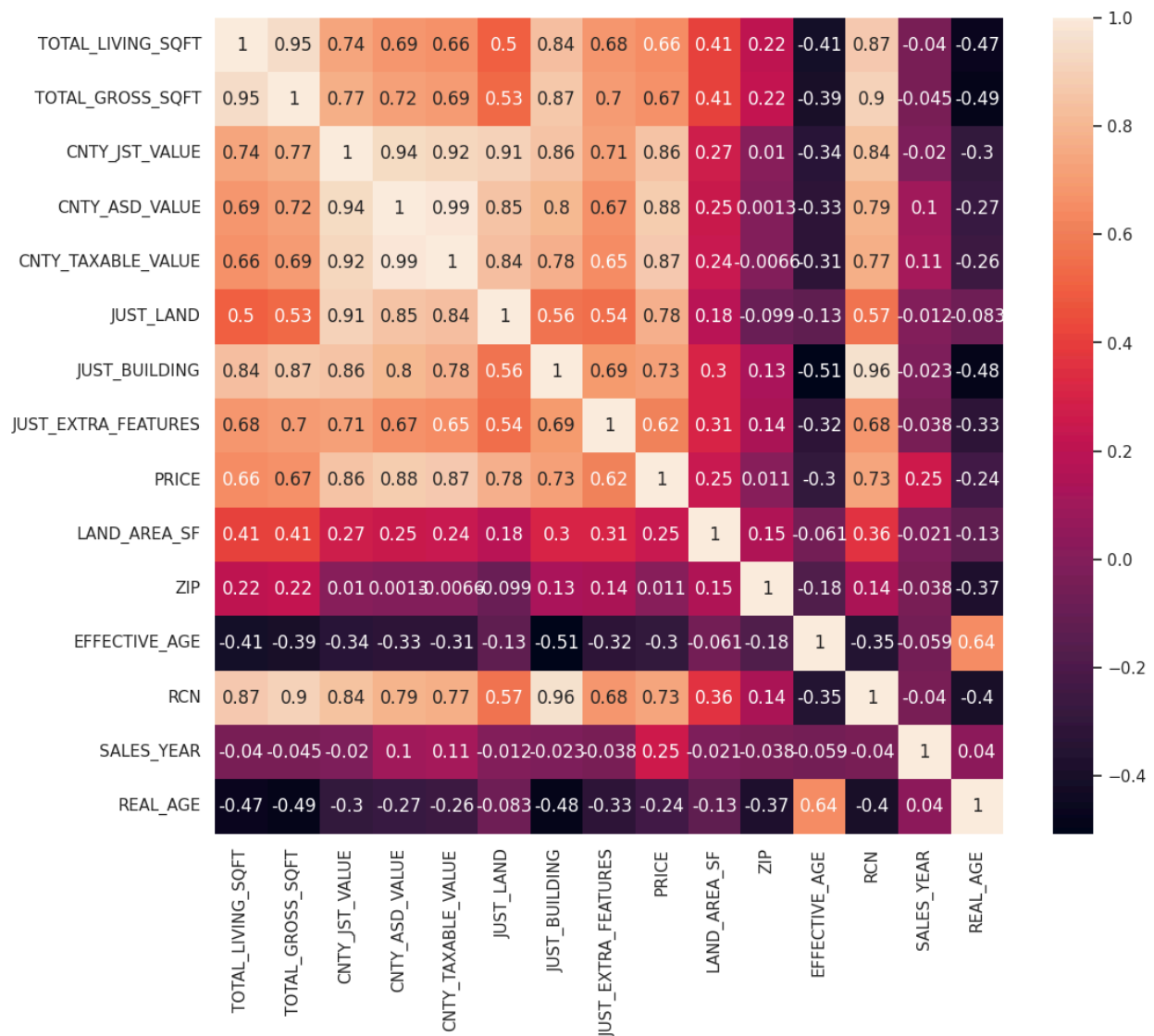
My dataset now had 218 thousand records with 32 columns. If I wanted the model to predict house prices, I wanted to have the model focus only on more recent sales. Finally the data was reduced to sales only from 2016 to July 2024. This was a total of 85 thousand rows exported into a CSV titled sales16_cleaned. A second dataset, PCPAO_SalesDATA_cleaned was also exported to CSV including all sales from 1970 to July 2024, but was never further used.

Exploratory Data Analysis

Subjecting the cleaned data to analysis, the house price distribution was examined and statistical tests of house features against price were done to determine which features to keep, condense or discard.

Numerical Features

A correlation matrix was run on the numerical features. Features like house size, assessed property, building, and land values, and reconstruction costs had moderate correlation with the sale price of the house.



The issue is that assessed property, building, and land values are recalculated every year by the county while the sale price is a value set in the past. It's not practical to predict current values by using future values, so many county-calculated values like the assessed value of the property, building, and land were dropped. The only numerical features kept for modeling were the house price, the total living area in square feet, the total gross area in square feet, the

land area in square feet, the effective age of the house, and the county's justified price of additional features as it was significantly smaller than the house price.

During the EDA, I wanted to find which features affected price the most. In regards to numerical features, the two features that had the largest effect on price were a house's TOTAL_LIVING_SQFT and TOTAL_GROSS_SQFT. Both had a .66 correlation with PRICE.

I also ran a linear regression of TOTAL_LIVING_SQFT and TOTAL_GROSS_SQFT on PRICE to find their prices per square foot. Using only TOTAL_LIVING_SQFT to predict PRICE gave a \$263 price per square foot and a price intercept of about -\$30,000. Doing the same thing for TOTAL_GROSS_SQFT gave a \$177 price per square foot and a price intercept of -\$63. Using the median TOTAL_LIVING_SQFT of 1512 sq. ft. and TOTAL_GROSS_SQFT of 2107 sq. ft. to calculate predicted median prices, I found a median price of \$368 thousand for the living square footage and a median price of \$373 thousand for the gross square footage. Both of these values were within 30-40 thousand of the true median sale price of \$335,000.

	Total Living Space in Sq. Ft.	Total Gross Space in Sq. Ft.
Price per Sq. Ft. from a linear regression	\$263 per Sq. Ft.	\$177 per Sq. Ft.
Median Area	1512 Sq. Ft.	2108 Sq. Ft.
Median Area x \$/Sq.Ft.	\$397656	\$373116
Intercept Price	- \$29758	- \$63
Predicted Price	\$367898	\$373053

I also created a prediction for price based on the median price per square foot. I found the median price per square foot of living space was \$217 and the median price per square foot of gross space was \$157. Using the same median areas as before, the median prices I found for living space and gross space were \$328 thousand and \$331 thousand.

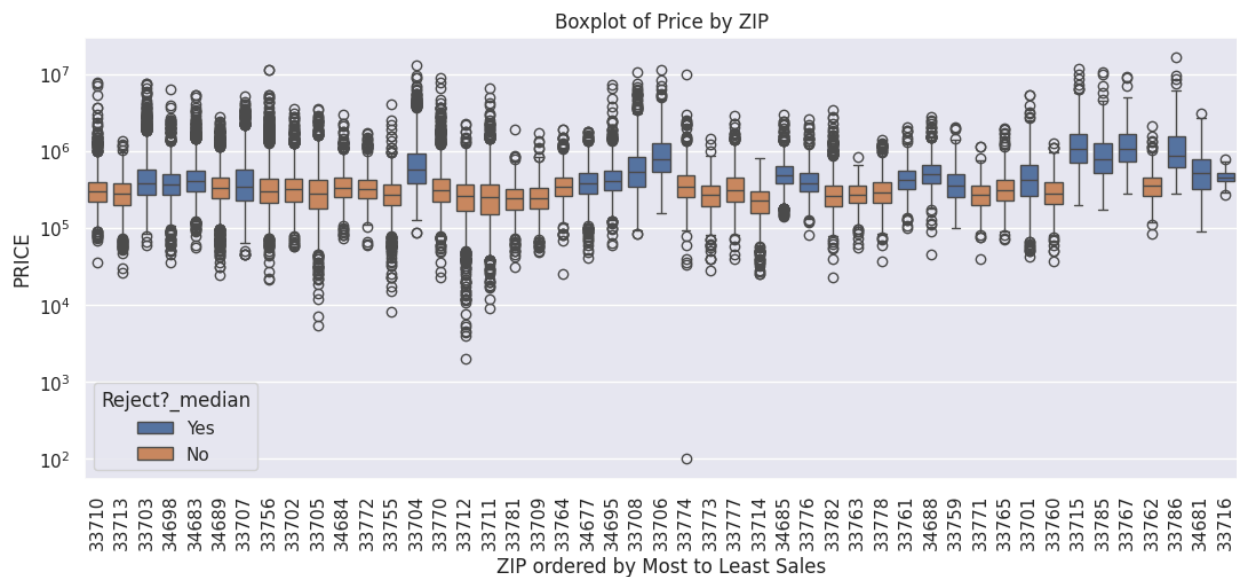
	Total Living Space in Sq. Ft.	Total Gross Space in Sq. Ft.
Median Price per Sq. Ft.	\$217	\$157
Median Area	1512 Sq. Ft.	2108 Sq. Ft.
Median Area x \$/Sq.Ft.	\$328104	\$330956

All of these prices based on the median living space and median gross space are pretty close to the median house price of \$335,000 for the entire county, and demonstrate that the TOTAL_LIVING_SPACE and TOTAL_GROSS_SPACE are effective predictors of PRICE.

Categorical Features

For each categorical feature value, I generated bootstrap-replicated sample statistics of the median property value to see if each categorical feature value had a positive statistically significant difference. I started with the categories with the most values, ZIP and CITY.

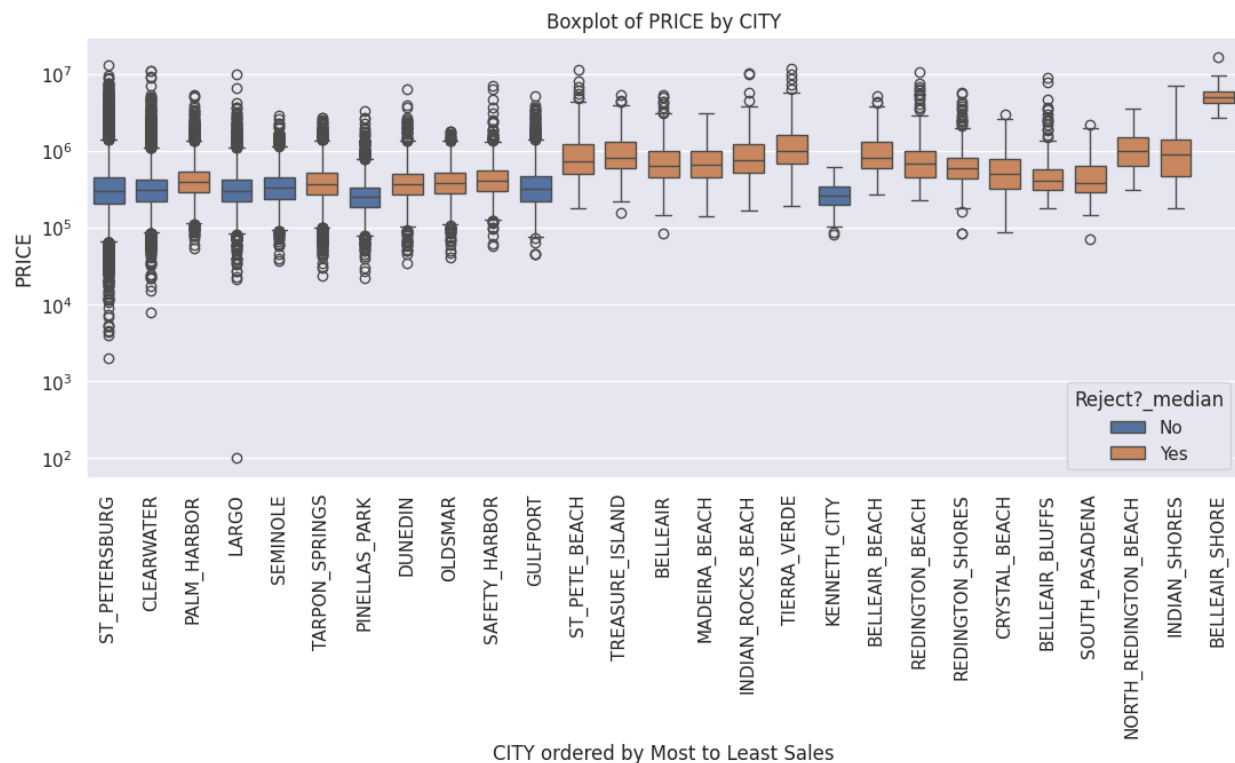
ZIP Code



I created a set of boxplots of house price by ZIP code, ordered from most sales to least sales, with each boxplot colored based on whether the median price in that ZIP was statistically significant from the county as a whole. Roughly half of the ZIP codes appeared as statistically significantly greater than the median. While ZIP code seems important for median property values, I did not see much of a pattern.

While there were statistically significant ZIP codes, because I didn't see any pattern in the ZIP codes I kept all of the ZIP code information later on in the preprocessing step. I did not reduce the number of ZIP codes.

City



I did the same set of boxplots of house price by CITY, ordered from most sales to least sales, with each boxplot colored based on whether the median price in that city was statistically significant from the county as a whole. With this sorting, I did see what I considered two important patterns.

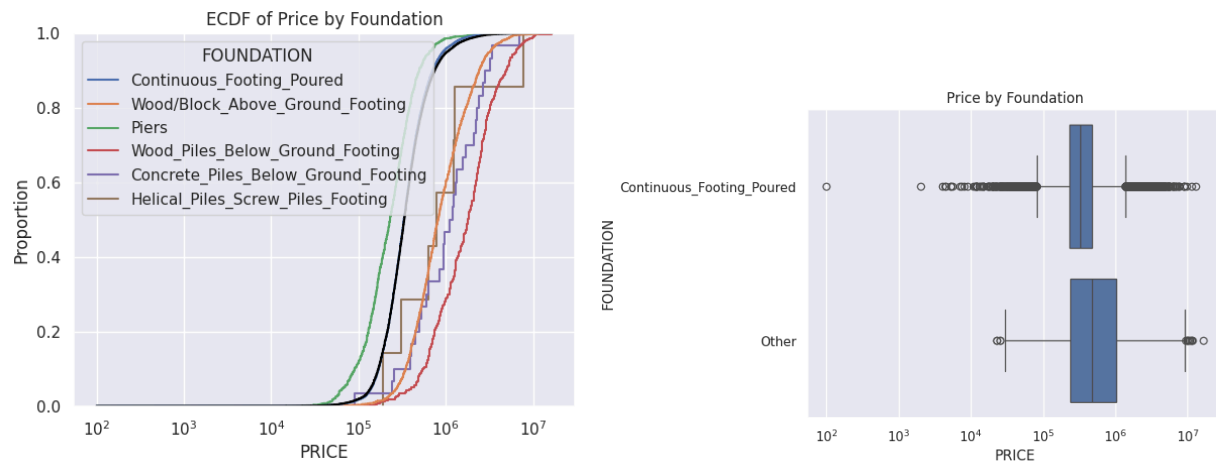
First, most of the beach towns had statistically significant medians. Every city colored orange in the plot above from ST_PETE_BEACH to the right is on the Gulf of Mexico. I found that the minimum difference in the median price of a house in one of those beach towns and the median price of a house in the county was \$50,000.

Second, those beach towns also accounted for under 5000 sales over 8 years. This works out to about only 42 single-family home sales per a year per a beach town. There is clearly a low supply of single-family home sales in the beach towns compared to other parts of the county, generating prices for those homes when on the market.

Since there was a pattern in the statistical significance of the smaller beach towns, I later condensed those towns into a single BEACH_TOWN value. I also changed KENNETH_CITY to join into the ST_PETERSBURG value as Kenneth City was a small number of occurrences compared to St. Petersburg.

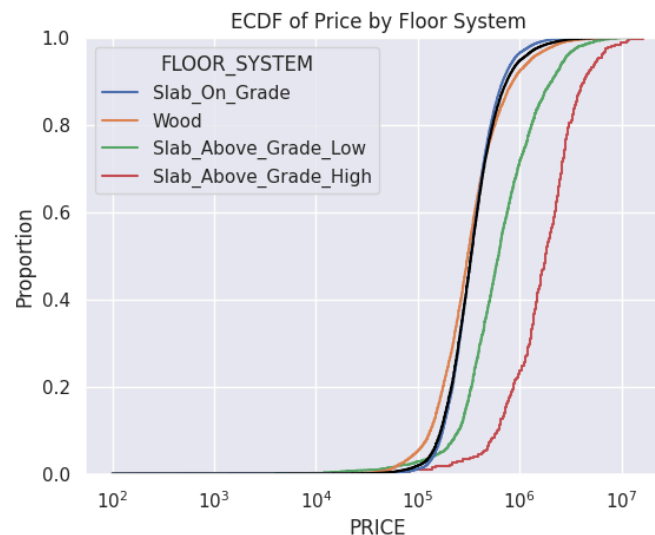
After exploring CITY and ZIP, I explored the house construction features, working from the FOUNDATION up to the ROOF_COVER.

Foundation



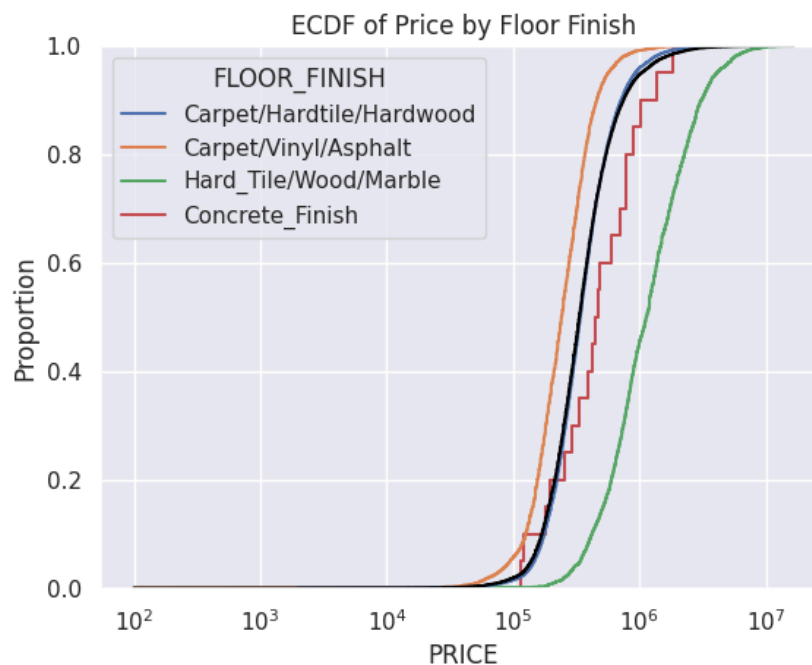
There were six different types of foundation for the houses. However, one foundation type accounted for 94% of the houses: the Continuous_Footing_Poured type. I opted to reduce the other 5 categories into an 'Other' type. Using the same statistical tests as before, I found that having a foundation type of 'Other' had a statistically significant effect on the median price. The median price of a house with a Continuous_Footing_Poured was \$330,000 while the median price of a house with an 'Other' foundation was \$469,000.

Floor System



There were four different types of floor system for the houses. Slab_on_Grade and Wood accounted for a strong majority of the floor systems used, roughly 97%. I found that having a floor system type of 'Slab_Above_Grade_Low' or 'Slab_Above_Grade_High' had a statistically significant effect on the median price. The median price of a house with a 'Slab_on_Grade' floor system was \$333,500. The median price of a house with a 'Wood' floor system was \$310,000. The median price of a house with a 'Slab_Above_Grade_High' floor system was \$1,800,000. The median price of a house with a 'Slab_Above_Grade_Low' floor system was \$620,000.

Floor Finish



There were four different types of floor finishes for the houses, after combining two feature values, 'Carpet/Vinyl/Asphalt/S' value and 'Carpet/_Vinyl/Asphalt', into a 'Carpet/Vinyl/Asphalt' value. The median price for houses with a 'Carpet/Hardtile/Hardwood' floor finish was \$340,000, with an IQR of \$248,000. This type of floor finish was the most common, with a count of 73,320 houses listed with that finish. The median price for a house with a 'Carpet/Vinyl/Asphalt' floor finish was \$241,500 with an IQR of \$175,000. This type of floor finish was the second most common, with a count of 9,321 houses listed with that finish. The median price for a house with a "Hard_Tile/Wood/Marble" finish was \$1,135,000 with an IQR of \$1,307,250. There were only 2,588 houses with this floor finish. Finally, there were 20 houses with a "Concrete_Finish" floor. The median house price for this type was \$460,850, with an IQR of \$500,000.

Exterior Walls

There were 10 different exterior wall categories. The largest category was "Cb_Stucco/Cb_Reclad" with about 51 thousand samples. The remaining 9 categories I recategorized into "Concrete_Block/Masonry" and "Frame". My reasoning is that the outside walls('EXTERIOR_WALLS') are either made of wood('Frame'), unclad block/stone ('Concrete_Block/Masonry') or covered block/stone('Cb_Stucco/Cb_Reclad'). "Concrete_Block/Masonry" had about 16 thousand samples and "Frame" had about 18 thousand samples. "Cb_Stucco/Cb_Reclad" houses had a median price of \$375,000 with an IQR of \$280,000. "Concrete_Block/Masonry" houses had a median price of \$265,000 with an IQR of \$170,000. "Frame" houses had a median price of \$305,000 with an IQR of \$260,000.

Interior Finish

There were 5 different interior finishes, but only two had significant sample sizes: “Drywall/Plaster” and “Upgrade.” I added the two smallest groups, “Mh_Average” and “Wood/Wallboard” to the “Drywall/Plaster” category. A “Custom” finish category was added to the “Upgrade” category. I joined the groups based on their means and standard deviations. With only two categories, “Drywall/Plaster” accounted for about 66 thousand samples with a median price of \$300,000 and an IQR of about \$200,000, while “Upgrade” accounted for about 19 thousand samples with median price of \$541,000 and an IQR of about \$470,000.

Roof Cover

There were initially 11 different roof covers, ranging from the standard composite shingles to solar-panel tiles. “Shingle_Composition” accounted for 69,560 samples. The next largest group was “Concrete_Tile/Metal” which accounted for 9,747 samples. I condensed the remaining 9 groups into an “Other” category which accounted for 5,942 samples.

“Shingle_Composition” houses had a median price of \$323,000 with an IQR of \$220,000.

“Concrete_Tile/Metal” houses had a median price of \$539,000 with an IQR of \$535,000.

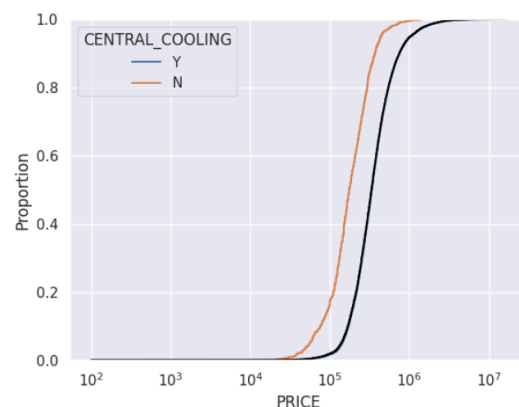
“Other” houses had a median price of \$288,000 with an IQR of \$280,000.

Roof Frame

There were 3 different roof frame types: “Flat_Shed”, “Gable_Or_Hip”, and “Mansard/Gambrel”. “Gable_Or_Hip” was the most common with 83,000 occurrences. The median price of a house with a “Gable_Or_Hip” roof frame was \$335,000 with an IQR of \$250,000. “Flat_Shed”-roofed houses made up only 1600 samples with a median price of \$320,000 and an IQR of \$330,000. There were only 135 houses with a “Mansard/Gambrel” roof frame. Those houses had a median price of \$490,000 and an IQR of \$450,000.

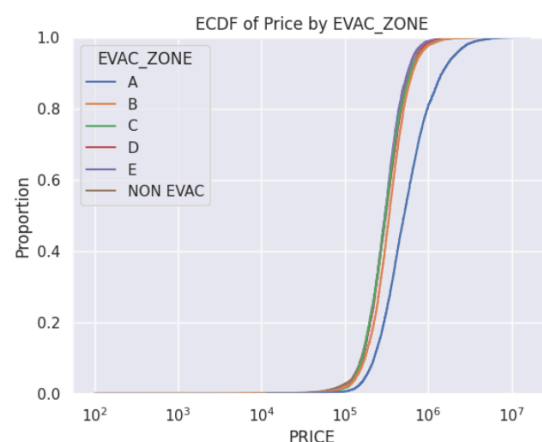
Central Cooling

The majority (83,878) of houses had central cooling. Those houses had a median price of \$337,000 and an IQR of \$260,000. The remaining 1,371 houses that had no central cooling had a median price of \$175,000 and an IQR of \$150,000.



Evacuation Zone

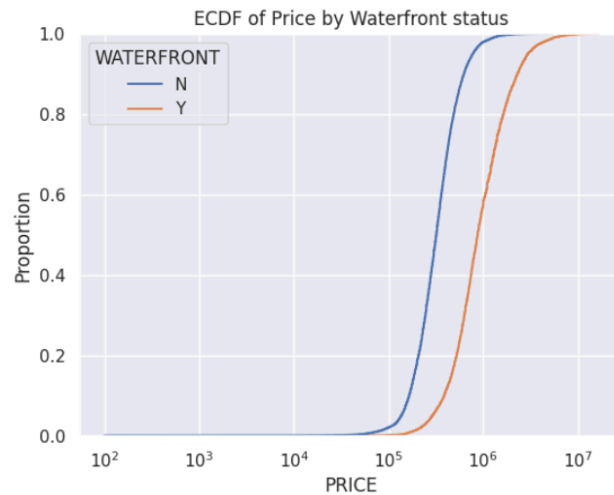
There were 6 evacuation zone codes: A, B, C, D, E, and NON EVAC. The only evacuation zone code that stood out from the rest based on price was zone A. Houses in evacuation zone A had a median price of \$515,000 with an IQR of \$525,000. The remaining houses in other



evacuation zones had median prices ranging from \$301,000 to \$340,000, with IQRs in the range of \$200,000.

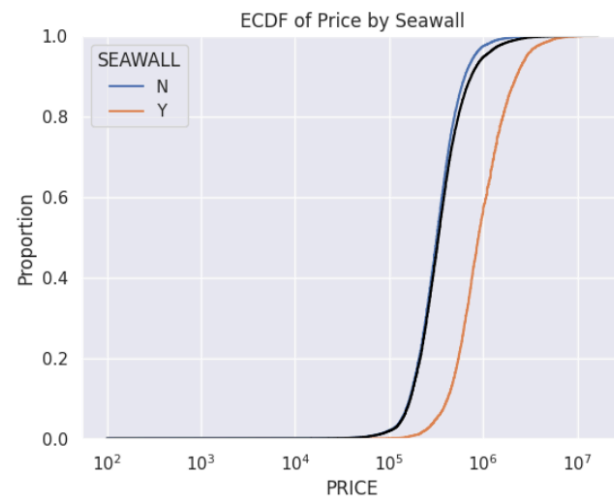
Waterfront Status

Waterfront status had a similar effect to house prices as an evacuation zone. There were 6798 houses with waterfront access that had a median price of \$875,000 and IQR of about \$800,000. The 78,451 houses with no waterfront access had a median price of \$320,000 and an IQR of \$220,000.



Seawall

There were 5695 houses with seawalls. Those houses had median prices of \$890,000 and an IQR of \$825,000. This was to be expected as there is significant overlap between waterfront status and needing a seawall. 79,554 houses without seawall had a median price of \$320,000 and an IQR of \$220,000.



Pool

I found the mean and median price of a house with a pool is greater than the mean and median price of a house without a pool. There were 30,000 houses with pools that had a median price of \$475,000 and an IQR of \$365,000. There were 55,000 houses with no pools that had a median price of \$280,000 and an IQR of \$180,000.

Subsidence

There was a "Subsidence" category with only 861 houses experiencing subsidence. Whether a house was experiencing any subsidence was not statistically significant to the median house price. This column was later dropped.

Preprocessing

Preprocessing consisted of 5 main steps:

1. Final reworking of the sales16_cleaned dataset
2. Cleaning and processing new data
3. Creating a Train-Test Split
4. Transforming numeric features
5. Transforming categorical features

Final Rework of sales16_cleaned

After all the EDA on the sales16_cleaned dataset, I needed to make sure any column and values changes were implemented. I started by dropping columns that were future-valued, were dates, or were not statistically significant.

```
#COLUMNS that had yearly updated values
future_cols = ['CNTY_JST_VALUE', 'CNTY_ASD_VALUE', 'CNTY_TAXABLE_VALUE', 'JUST_LAND', 'JUST_BUILDING', 'RCN']
#columns that are dates
time_cols = ['SALES_DATE', 'SALES_YEAR']
#columns with little significance
unused_cols = ['REAL_AGE', 'CONTAMINATION', 'SUBSIDENCE']
sales_16.drop(columns=future_cols+time_cols+unused_cols, inplace=True)
```

I also had to convert the ZIP column from numerical to object. Then I converted all object-typed columns to string-type columns.

```
sales_16.ZIP = sales_16.ZIP.astype('object')

for col in sales_16.select_dtypes(include='object').columns:
    sales_16[col] = sales_16[col].astype('string')
```

I had in earlier cleaning replaced all spaces in city names with underscores. I made the choice here to do the same thing to the “NON EVAC” value in the “EVAC_ZONE” column.

```
sales_16['EVAC_ZONE'] = sales_16['EVAC_ZONE'].str.replace('NON EVAC', 'NON_EVAC')
```

Cleaning and Processing of New Data

At the time of preprocessing, I decided to collect some new data for home sales from July 2024 to December 2024. I made sure to collect the same type of information from the PCPAO website as I did before. This data set was essentially held in reserve but needed the same sort of cleaning that the sales_16 data received. It was 3877 rows, with some duplicate rows and some parcels sold more than once.

The only change between records on parcels that sold more than once were in the “PRICE” column. The highest price of each parcel was kept and duplicate rows were dropped, leading to only 3767 rows. Dropping any rows with null values brought the number down to 3765.

The remaining cleaning steps of the new data involved column name changes to match the sales_16 dataset and cleaning string values to replace spaces with underscores, and

making sure string values for “Yes” and “No” were turned into “Y” and “N” like in the sales_16 dataset. This dataset was referred to as final_sales.

Creating a Train-Test Split

I took the sales_16 dataset and the final_sales dataset and split them into X_sales16, y_sales16, X_final, and y_final to separate the features from the target. I then used test_train_split to create X16_train, X16_test, y16_train, y16_test from the X_sales16 and y_sales16 datasets.

```
X_sales16 = sales_16.drop(columns='PRICE')
y_sales16 = sales_16['PRICE']
X_final = final_sales.drop(columns='PRICE')
y_final = final_sales['PRICE']
```

```
X16_train, X16_test, y16_train, y16_test = train_test_split(X_sales16, y_sales16, test_size=0.2, random_state=42)
```

Transforming Numeric Features

I created a set of numeric datasets from the X16_train, X16_test and X_final datasets so that I could complete any transformations on the numeric datasets.

```
numeric_feat = ['TOTAL_LIVING_SQFT', 'TOTAL_GROSS_SQFT', 'JUST_EXTRA_FEATURES', 'LAND_AREA_SF', 'EFFECTIVE_AGE']
#creating dataframes of the numeric features
numeric_X16_train = X16_train[numeric_feat]
numeric_X16_test = X16_test[numeric_feat]
numeric_final = X_final[numeric_feat]
```

Inspecting the histograms of the numeric datasets, all of the features were skewed except for EFFECTIVE_AGE. I started with log-scaling each feature in the datasets. I created a PowerTransformer object called log. I then applied log.fit_transform() to the numeric_X16_train dataset and log.transform() to the numeric_X16_test and numeric_final datasets. The transformed features were saved in new columns with “log_” prepended to the feature names.

```
for col in ['TOTAL_LIVING_SQFT', 'TOTAL_GROSS_SQFT', 'LAND_AREA_SF', 'JUST_EXTRA_FEATURES']:
    log = PowerTransformer()
    numeric_X16_train[str('log_'+col)] = log.fit_transform(numeric_X16_train[[col]])
    numeric_X16_test[str('log_'+col)] = log.transform(numeric_X16_test[[col]])
    numeric_final[str('log_'+col)] = log.transform(numeric_final[[col]])
```

```
numeric_X16_train.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 68199 entries, 2229 to 15795
Data columns (total 9 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   TOTAL_LIVING_SQFT                    68199 non-null  float64
1   TOTAL_GROSS_SQFT                    68199 non-null  float64
2   JUST_EXTRA_FEATURES                 68199 non-null  int64
3   LAND_AREA_SF                        68199 non-null  int64
4   EFFECTIVE_AGE                       68199 non-null  float64
5   log_TOTAL_LIVING_SQFT               68199 non-null  float64
6   log_TOTAL_GROSS_SQFT               68199 non-null  float64
7   log_LAND_AREA_SF                   68199 non-null  float64
8   log_JUST_EXTRA_FEATURES             68199 non-null  float64
dtypes: float64(7), int64(2)
memory usage: 5.2 MB
```

```
numeric_X16_test.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 17050 entries, 39481 to 70617
Data columns (total 9 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   TOTAL_LIVING_SQFT                    17050 non-null  float64
1   TOTAL_GROSS_SQFT                    17050 non-null  float64
2   JUST_EXTRA_FEATURES                 17050 non-null  int64
3   LAND_AREA_SF                        17050 non-null  int64
4   EFFECTIVE_AGE                       17050 non-null  float64
5   log_TOTAL_LIVING_SQFT               17050 non-null  float64
6   log_TOTAL_GROSS_SQFT               17050 non-null  float64
7   log_LAND_AREA_SF                   17050 non-null  float64
8   log_JUST_EXTRA_FEATURES             17050 non-null  float64
dtypes: float64(7), int64(2)
memory usage: 1.3 MB
```

While PowerTransformer automatically standardizes its output values, I still needed to standardize the non-log-scaled values. I used SciKit-learn's StandardScaler on all the numeric features.

```
numeric_cols = ['TOTAL_LIVING_SQFT', 'TOTAL_GROSS_SQFT', 'JUST_EXTRA_FEATURES', 'LAND_AREA_SF', 'EFFECTIVE_AGE']
log_cols = ['log_TOTAL_LIVING_SQFT', 'log_TOTAL_GROSS_SQFT', 'log_JUST_EXTRA_FEATURES', 'log_LAND_AREA_SF']
for col in numeric_cols+log_cols:
    scaler = StandardScaler()
    numeric_X16_train[col] = scaler.fit_transform(numeric_X16_train[[col]])
    numeric_X16_test[col] = scaler.transform(numeric_X16_test[[col]])
    numeric_final[col] = scaler.transform(numeric_final[[col]])
    print(col)
    print(scaler.mean_)
    print(scaler.scale_)
    print()
```

Transforming Categorical Features

I started by creating categorical feature datasets from the X16_train, X16_test, and X_final datasets.

```
cat_X16_train = X16_train.select_dtypes(include='string')
cat_X16_test = X16_test.select_dtypes(include='string')
cat_final = final_sales.select_dtypes(include='string')
```

With these categorical datasets, I had some concern that there might be unique values in one dataset that would cause issues when dummy-encoding and one-hot-encoding later. I did differences of sets to find if any dataset had a unique value not in the other datasets.

```
for col in cat_X16_train.columns:
    print(col)
    print('Unique values in cat_X16_train', set(cat_X16_train[col].unique())-set(cat_final[col].unique())-set(cat_X16_test[col].unique()))
    print('Unique values in cat_X16_test', set(cat_X16_test[col].unique())-set(cat_final[col].unique())-set(cat_X16_train[col].unique()))
    print('Unique values in cat_final', set(cat_final[col].unique())-set(cat_X16_train[col].unique())-set(cat_X16_test[col].unique()))
    print()
```

I used this information to make changes in the cleaning of the sales_final dataset earlier and to inform on any category reductions that take place later.

For binary features like whether a house had a pool, seawall, central cooling, or was on the water, I used one-hot-encoding, dropping the first value in the feature.

```
ohe = OneHotEncoder(drop='first', sparse_output=False)
bin_cat_X16_train = ohe.fit_transform(cat_X16_train[binary_cols])
bin_cat_X16_test = ohe.transform(cat_X16_test[binary_cols])
bin_cat_final = ohe.transform(cat_final[binary_cols])
```

As the OneHotEncoder transform returns a numpy array, I had to create a new pandas dataframe for the transformed results.

```
bin_cat_X16_train_df = pd.DataFrame(bin_cat_X16_train, columns=ohe.get_feature_names_out(), index=train_index_values)
bin_cat_X16_test_df = pd.DataFrame(bin_cat_X16_test, columns=ohe.get_feature_names_out(), index=test_index_values)
bin_cat_final_df = pd.DataFrame(bin_cat_final, columns=ohe.get_feature_names_out(), index=final_index_values)
```

As I progressed through transforming the categorical features, I concatenated the categorical features with the numeric features in a set of preprocessed dataframes.

```
preprocessed_X16_train = pd.concat([numeric_X16_train, bin_cat_X16_train_df], axis=1)
preprocessed_X16_test = pd.concat([numeric_X16_test, bin_cat_X16_test_df], axis=1)
preprocessed_final = pd.concat([numeric_final, bin_cat_final_df], axis=1)
```

My basic process for continuing to transform the categorical features was this.

1. Complete any relabeling of values based on EDA(if necessary)
2. Create a OneHotEncoder for the category. Fit the encoder on the feature of interest using the cat_X16_train dataset.
 - a. For the construction features, I dropped the most common value in each feature, making the construction features dummy-encoded technically.
 - b. For other features, I dropped no values.
3. Make three new dataframes for the transformed feature: one each for cat_X16_train, cat_X16_test, and cat_final.
4. Concatenate the new dataframes onto the preprocessed_X16_train, preprocessed_X16_test, and preprocessed_final datasets.

I did at the end turn the EVAC_ZONE feature into a binary feature focused on whether the house was in EVAC_ZONE A as that was the only value that had house prices statistically significantly different from the median.

After all of the categorical feature transformations, I had 6 datasets to export as CSV files: the 3 preprocessed dataframes and the 3 y value dataframes.

Model Selection

To create a price prediction model, I tried out a variety of regressor models. To determine which model was performing best, I settled on using a Mean Absolute Percentage Error metric. My reasoning for this metric was the MAPE would give an error in a percentage applicable to all houses as opposed to an error in dollars which would not apply to every house. A model's MAPE of 10% would say that any predicted price is 10% away from the true price on average. Using another error like root-mean-squared error would give an error predicted in dollars. If the predicted error from RMSE was say \$50,000, that error on a house could be 20% error on a \$250,000 house, 10% error on a \$500,000 house or 1% on a \$5,000,000 house.

Baseline Dummy Regressors

I started by getting some basic statistics on how two dummy regressor models would perform. I created a dummy regressor to predict the mean and a dummy regressor to predict the median. I found their R-Squared Scores, RMSE and MAPE for both the train and test sets.

DummyRegressor: Mean strategy
 TRAINING SCORE
 R-Squared Score: 0.0
 RMSE: 446422.66846135724
 MAPE: 0.7408686291069302

TESTING SCORE
 R-Squared Score: -4.3856965126209957e-07
 RMSE: 420910.45743294875
 MAPE: 0.9986209112817827

DummyRegressor: Median strategy
 TRAINING SCORE
 R-Squared Score: -0.05220732771521974
 RMSE: 457927.68453478464
 MAPE: 0.5239374218305217

TESTING SCORE
 R-Squared Score: -0.058407394278028324
 RMSE: 433028.07387058577
 MAPE: 0.7228231769693985

Focusing on the MAPE results, the median-trained dummy regressor performed better than the mean-trained dummy regressor.

Linear Regressor

Proceeding on to a linear regressor model, I ran the same fit scoring of the model as dummy regressors. The linear regressor obviously improved upon the dummy regressor, with the MAPE decreasing in both the training and testing sets. Along the way I created a function to evaluate my model's performance in different price ranges.

True Values Between 0 and 500000
Size of sample: 13019
RMSE: 133242.04693847452
MAE: 90327.5533176215
MAPE: 0.8296285301235278
R-Squared: -0.6197009860543394
Accuracy: 0.21115293033259083

True Values Between 500000 and 1000000
Size of sample: 3129
RMSE: 260093.13611961514
MAE: 204748.45386063014
MAPE: 0.3102103643680071
R-Squared: -2.866536937581308
Accuracy: 0.16267178012144456

I monitored each model's performance along the way in different price ranges, as I was interested in what ranges the model performed best at. The linear regressor's MAPE was best in the \$500,000 to \$5,000,000 range.

Decision Tree

After fitting and creating a decision tree model, I got an obvious improvement in MAPE score on the training set and a worse score on the test set. When inspecting the Decision Tree regressor's model performance on the same ranges as the linear regressor on the test set, the Decision Tree regressor's MAPE was higher in every category.

Random Forest Regressor

Using the default setting for the Random Forest Regressor, I did see improvement compared to the linear regressor and the decision tree. As you would expect, the MAPE on the training set went up compared to the decision tree. This is because the decision tree was essentially

LinearRegressor

TRAINING SCORE

R-Squared Score: 0.6315971775321281

RMSE: 270961.57841908874

MAPE: 0.3796796065032334

TESTING SCORE

R-Squared Score: 0.6413760832177333

RMSE: 252063.08387303507

MAPE: 0.7056439145595768

True Values Between 1000000 and 5000000

Size of sample: 888

RMSE: 657879.9274601501

MAE: 488910.1170069108

MAPE: 0.28328363818361585

R-Squared: 0.14981395951466991

Accuracy: 0.16554054054054054

True Values Between 5000000 and 20000000

Size of sample: 14

RMSE: 4277429.056763415

MAE: 3987178.4491342125

MAPE: 0.577914910573381

R-Squared: -9.799166744172064

Accuracy: 0.0

DecisionTreeRegressor

TRAINING SCORE

R-Squared Score: 0.9999579778672679

RMSE: 2893.9117570266967

MAPE: 0.0001535620778641294

TESTING SCORE

R-Squared Score: 0.25639694142622815

RMSE: 362961.20185847365

MAPE: 0.7811497656564296

overfitting to the training set. Because the decision tree was overfitting, its performance on the test set was worse than the linear regressor. The random forest regressor's use of a variety of decision trees meant it could not overfit as badly as the lone decision tree regressor. The random forest regressor's MAPE on the training set and test set showed improvement over the linear regressor.

During the review of the random forest regressor, I did inspect what the model found as the top 10 most important features. It found the 8 of the 10 most important features were numeric features, either log-scaled or not log-scaled. The 2 categorical features of importance were whether the house was on the waterfront and if the house floor finish was hard wood, tile, or marble.

	Feature_Importance
log_TOTAL_GROSS_SQFT	0.180910
TOTAL_GROSS_SQFT	0.156180
WATERFRONT_Y	0.101826
log_TOTAL_LIVING_SQFT	0.094979
TOTAL_LIVING_SQFT	0.090892
FLOOR_FINISH_Hard_Tile/Wood/Marble	0.064752
EFFECTIVE_AGE	0.050117
log_JUST_EXTRA_FEATURES	0.031678
JUST_EXTRA_FEATURES	0.030011
log_LAND_AREA_SF	0.028694

AdaBoost Regressor

Using a default AdaBoost regressor, which runs a decision tree regressor with a depth of 3 and boosts the performance, it still underperformed compared to the linear regressor or the random forest regressor. I also tried an AdaBoost regressor using a linear

RandomForestRegressor
 TRAINING SCORE
 R-Squared Score: 0.9530418137758414
 RMSE: 96739.0673301809
 MAPE: 0.1256476147101167
 TESTING SCORE
 R-Squared Score: 0.6534855850860872
 RMSE: 247770.88583753788
 MAPE: 0.6810258579853672

AdaBoostRegressor
 TRAINING SCORE
 R-Squared Score: 0.4482568465803608
 RMSE: 331600.1478061928
 MAPE: 0.6116630154301882
 TESTING SCORE
 R-Squared Score: 0.4001117140353979
 RMSE: 326005.4131594914
 MAPE: 0.949604720571543

regressor that was boosted. That model performed worse than the dummy regressors in both training and testing sets.

Histogram-Based Gradient Boosting Regression Tree

I made use of this model type over Sci-Kit Learn's normal Gradient Boosting Regression Tree because documentation suggested this type of model if you have over 10,000 samples of data. The HGB Regressor model's performance on the training score compared to the linear regressor was impressive. The model's MAPE on the test set was also an improvement to the random forest regressor and the linear regressor.

When inspecting the HGB-regressor's MAPE performance on the test set in a set of price ranges, the MAPE was lower in each range than the random forest regressor.

True Values Between 0 and 500000
Size of sample: 13019
RMSE: 112490.749296337
MAE: 84452.66834595676
MAPE: 0.7990410815727701
R-Squared: -0.1544785651273417
Accuracy: 0.18227206390659806

True Values Between 500000 and 1000000
Size of sample: 3129
RMSE: 229994.70657748904
MAE: 181639.37203897134
MAPE: 0.270283897620406
R-Squared: -2.0234310034530125
Accuracy: 0.16075423457973795

HistGradientBoostingRegressor

TRAINING SCORE

R-Squared Score: 0.7476109052934548

RMSE: 224275.3442882672

MAPE: 0.362106970643085

TESTING SCORE

R-Squared Score: 0.6801730819947429

RMSE: 238038.45737266025

MAPE: 0.6767204543708423

True Values Between 1000000 and 5000000

Size of sample: 888

RMSE: 741941.464311522

MAE: 547280.8924043125

MAPE: 0.3195256367608696

R-Squared: -0.08133431181331141

Accuracy: 0.12837837837837837

True Values Between 5000000 and 20000000

Size of sample: 14

RMSE: 3240453.4219626933

MAE: 2873047.1419413346

MAPE: 0.4222017806473364

R-Squared: -5.197780382247677

Accuracy: 0.0

XGBoost Regressors

I finally tried out two different XGBoost regressors. The first was a XGBRegressor that boosts a decision tree through 10 rounds of improvement. Its performance was comparable to the HGB Regressor that I previously trained. The second was a XGBRF Regressor that boosts a random forest model through 10 rounds of improvement. It did not perform as well as the regular XGB Regressor or the HGB Regressor. As the HGB Regressor performed just as well as the XGBoost regressor and it was part of the Sci-Kit learn library, I opted to just focus on doing hyperparameter tuning on the HGB regressor.

XGBRFRegressor	XGBRegressor
TRAINING SCORE	TRAINING SCORE
R-Squared Score: 0.6802074313163757	R-Squared Score: 0.7576828002929688
RMSE: 252452.9375	RMSE: 219754.796875
MAPE: 0.39372187852859497	MAPE: 0.3629192113876343
TESTING SCORE	TESTING SCORE
R-Squared Score: 0.6428060531616211	R-Squared Score: 0.6617206931114197
RMSE: 251560.046875	RMSE: 244808.984375
MAPE: 0.7039985656738281	MAPE: 0.6769898533821106

Hyperparameter Tuning

I created 3 different HGB regressors and ran each regressor through a randomized-search cross validation, having the models try out different parameters in their training. The parameters I tried out in the first RandomizedSearchCV were loss function, learning rate, maximum number of iterations, maximum number of leaf nodes, maximum tree depth, minimum number of samples per leaf, and L2 regularization. In later runs of the RandomizedSearchCV, I also included maximum features used as a parameter.

```
hgb_reg = HistGradientBoostingRegressor(scoring='neg_mean_absolute_percentage_error')
params = {
    'loss': ['squared_error', 'absolute_error', 'poisson'],
    'learning_rate': [0.01, 0.05, .1],
    'max_iter': [50, 150, 300, 500],
    'max_leaf_nodes': [10, 50, 100],
    'max_depth': [3, 5, 7, 15],
    'min_samples_leaf': [10, 20, 50, 100],
    'l2_regularization': [0, 0.05, 0.1]
}
clf_1 = RandomizedSearchCV(hgb_reg, params, n_iter=40, random_state=11, n_jobs=-1)
clf_1.fit(X_train, np.ravel(y_train))

hgb_reg = HistGradientBoostingRegressor(scoring='neg_mean_absolute_percentage_error')
params = {
    'loss': ['squared_error', 'absolute_error', 'poisson'],
    'learning_rate': [0.01, 0.05, .1],
    'l2_regularization': [0, 0.05, 0.1],
    'max_iter': [100, 150, 200, 250, 300, 500, 1000],
    'max_depth': [None, 3, 5, 7, 15],
    'max_leaf_nodes': [10, 50, 100, 200, 500],
    'min_samples_leaf': [10, 20, 40, 100, 150],
    'max_features': [.25, .5, .75, 1.0]
}
clf_2 = RandomizedSearchCV(hgb_reg, params, n_iter=40, random_state=12, n_jobs=-1)
clf_2.fit(X_train, np.ravel(y_train))
```

After running the 3 RandomizedSearchCVs and getting each run's best parameters and scores, I was able to make the following table.

	HGB Regressor 1	HGB Regressor 2	HGB Regressor 3
categorical_features	from_dtype	from_dtype	from_dtype
early_stopping	auto	auto	auto
interaction_cst	None	None	None
l2_regularization	0.05	0.0	0.05
learning_rate	0.1	0.05	0.05
loss	poisson	poisson	poisson
max_bins	255	255	255
max_depth	5	7	7
max_features	1.0	0.75	0.5
max_iter	150	1000	1000
max_leaf_nodes	100	50	10
min_samples_leaf	100	150	150
monotonic_cst	None	None	None
n_iter_no_change	10	10	10
quantile	None	None	None
random_state	None	None	None

scoring	neg_mean_absolute_percentage_error	neg_mean_absolute_percentage_error	neg_mean_absolute_percentage_error
tol	0.0	0.0	0.0
validation_fraction	0.1	0.1	0.1
verbose	0	0	0
warm_start	False	False	False
mean_test_score	0.676922	0.683587	0.682327
std_test_score	0.024512	0.019758	0.019449

Results

I found the first HGB Regressor found by RandomizedSearchCV to perform the best. I evaluated the model at different price ranges and found the model performed with an MAPE between 25% and 40% in the range of prices between \$500,000 and \$20,000,000. The worst performance was always between \$0 and \$500,000, where I was finding 75%+ error. I suspected that some low-value house was causing significant error in the \$0 to \$500,000 range.

I confirmed that there was an issue here by evaluating the model's MAPE on a different range of values. Instead of \$0-\$500,000, I adjusted to measure performance on a range from \$50,000 to \$500,000. The model's MAPE in the range fell to 32%, much more in line with what the model found in other price ranges. At this time I pickled the model to save its parameters.

With the model parameters now saved, I combined the training and test sets into one full dataset and fit the model with all of the data. I scored the model on the full dataset and the secondary dataset that was held aside. That is when I found the worst "training" score from the full dataset

```
True Values Between 0 and 500000
Size of sample: 13019
RMSE: 107199.89712769497
MAE: 79949.02498602118
MAPE: 0.7718069773575275
R-Squared: -0.04843374774683573
Accuracy:0.1937168753360473
```

```
True Values Between 50000 and 500000
Size of sample: 12968
RMSE: 106584.1413461297
MAE: 79534.87102462503
MAPE: 0.327715872648004
R-Squared: -0.05740085231602121
Accuracy:0.1944787168414559
```

```
TRAINING SCORE
MAPE: 0.3940151528838157
TESTING SCORE
MAPE: 4.949755489307261
```

and even worse scores from the secondary dataset. The 490% MAPE in the testing set told me something was up. I found that the full dataset and final dataset each had a couple of houses with \$100 sale prices. Just by removing those \$100 sales, the MAPE improved.

True Values Between 0 and 500000	True Values Between 100 and 500000
Size of sample: 2244	Size of sample: 2239
RMSE: 117981.32757808405	RMSE: 116256.71300254994
MAE: 100429.45733903049	MAE: 99872.94617489848
MAPE: 8.08610824404715	MAPE: 0.29632383554040237
R-Squared: -0.5521140905886441	R-Squared: -0.5503582922787147
Accuracy: 0.11586452762923351	Accuracy: 0.11612326931665923

I dropped the \$100 sales from both dataset, refit the model on the full dataset and found the MAPE for the model on both the full and final datasets. The training and testing MAPE values both fell.

TRAINING SCORE
 MAPE: 0.3324150054730581
 TESTING SCORE
 MAPE: 0.3074643265855423

Comparable Houses

One last thing I did was to find a house on the market and see if my model could give a reasonable price range for the house that included what the house was listed for. The example house I found had a total living area in square feet of 2290, and a 10 thousand square foot lot. It had a pool, a seawall, was on the waterfront, and was in an A flood zone. By finding the houses that are similar, I could see if the asking price of 1.09 million dollars was in the range of prices for similar houses. I also checked against the zip code(33703) and city(Saint Petersburg) to help narrow down possible prices.

I scaled all of the features that I knew to write a query to find houses similar to my example house in the full dataset. I found 11 similar houses in the dataset and ran their features through the model to see what the model thought their price would be.



I found that while the true prices ranged from about 0.5 million dollars to 1.4 million dollars, the predicted prices ran in a much smaller range from about 0.8 million

dollars to 1.1 million dollars. The listed price of the house fit well within the range of predicted prices.

Possible Improvements

Improvements to the model could be made by turning the preprocessing and model selection into a pipeline. During the preprocessing, numerical features were scaled with a standard scaler, which uses the mean and standard deviation to place everything around 0. As mean house price can be affected by high priced outliers, having a model try using the sci-kit learn RobustScaler might give a model performance improvement as the RobustScaler scales using the median and IQR.